

MAT3120.3

Machine Learning and Data Visualisation

**Assignment 3: Report on The Analysis and
Modelling of a Dataset**

Student Name: L B Asmitha Darani Balasooriya

Student ID: 10687611

Table of Contents

PART 1: DATA PREPARATION AND CLEANING	3
PART 2: MODEL TRAINING AND HYPERPARAMETER TUNING.....	3
DEFINING THE PERFORMANCE METRICS IN THE CONTEXT OF THE STUDY	3
MODELLING APPROACH 1: LOGISTIC LASSO REGRESSION	3
<i>Hyperparameter Tuning Strategy</i>	3
<i>Cross-Validation Results</i>	4
<i>Performance Evaluation on Test Set</i>	4
MODELLING APPROACH 2: CLASSIFICATION TREE	5
<i>Hyperparameter Tuning Strategy</i>	5
<i>Cross-Validation Results</i>	5
<i>Performance Evaluation on Test Set</i>	6
MODELLING APPROACH 3: BAGGING TREE.....	6
<i>Hyperparameter Tuning Strategy</i>	6
<i>Cross-Validation Results</i>	7
<i>Performance Evaluation on Test Set</i>	7
RECOMMENDED MODEL AND CONCLUSION	8
<i>Comparative Performance Analysis</i>	8
<i>Justification for the Recommended Model</i>	8
<i>Conclusion</i>	8
REFERENCES	9
APPENDICES	10
APPENDIX A: MODEL SELECTION CONSOLE OUTPUT	10
APPENDIX B: CONFUSION MATRIX AND STATISTICS CONSOLE OUTPUT – LASSO REGRESSION.....	10
APPENDIX C: CONFUSION MATRIX AND STATISTICS CONSOLE OUTPUT – CLASSIFICATION TREE.....	10
APPENDIX D: CONFUSION MATRIX AND STATISTICS CONSOLE OUTPUT – BAGGING TREE.....	10

Table of Tables

Table 1 – Top 5 CV Results – Logistic LASSO Regression	4
Table 2 – Confusion matrix – Logistic LASSO Regression.....	4
Table 3 – Top 5 CV Results – Classification Tree	5
Table 4 – Confusion matrix – Classification Tree	6
Table 5 - Top 5 CV Results – Bagging Tree	7
Table 6 – Confusion matrix – Bagging Tree	7
Table 7 – Comparing 3 ML Models.	8

Table of Figures

Figure 1– ROC against Regularization Parameter Graph.....	4
Figure 2 – ROC against CP Graph.....	6

Part 1: Data Preparation and Cleaning

To ready wacy_data for trustworthy analysis and modeling, extensive cleaning processes were employed to handle missing data, simplify complexity, and improve feature distributions.

Dealing with Missing and Inconsistent Data - Missing Source.IP.Type.Detected and -1 in Attack.Source.IP.Address.Count were replaced with NA to represent missing or not applicable data consistently. Unreasonable values—Hits values ≤ 0 and 99999 in Average.ping.to.attacking.IP.milliseconds (likely placeholders)—were dropped. Finally, all NA rows were dropped to have complete data to train stable models.

Categorical Value Simplification - Merging categories helped reduce sparsity, enhance interpretability, and ensure generalization. For example, “Windows 10” and “Windows Server 2008” were grouped as Windows_All in Source.OS.Detected, reflecting similar OS types and likely attack patterns. In Target.Honeypot.Server.OS, “Windows (Desktops)” and “Windows (Servers)” were combined as Windows_DeskServ, while “Linux” and “MacOS (All)” were grouped into MacOS_Linux—both cases reflecting logical OS groupings to simplify learning and improve model performance.

Data Transformations - The Average.ping.variability was log-transformed in order to decrease skewness and to stabilize the variance, and square root of Hits, Attack.Source.IP.Address.Count, Average.ping.to.attacking.IP.milliseconds, and Individual.URLs.requested was calculated. These are in support of model precision since they normalize skewed distributions and facilitate algorithmic assumptions about structure. Collectively, these steps created a cleaned dataset (WACY_COM_cleaned) which was both statistically solid and analysis-ready, and well-organized.

Part 2: Model Training and Hyperparameter Tuning

Defining the Performance Metrics in the context of the Study.

Sensitivity (The APT Detection Rate) - Sensitivity, also known as the **True Positive Rate**, measures the proportion of actual positive cases that are correctly identified. In this study, it answers the critical question: "Of all the actual APT attacks that occurred, what percentage did our model successfully detect?"

Specificity (The Non-APT Detection Rate) - Specificity, or the **True Negative Rate**, measures the proportion of actual negative cases that are correctly identified. In this context, it answers: "Of all the benign (non-APT) network activities, what percentage did our model correctly identify as safe?"

Overall Accuracy - Accuracy is the proportion of all predictions that were correct, including both True Positives and True Negatives. It provides a general score of the model's correctness across all classes.

Modelling Approach 1: Logistic LASSO Regression

Hyperparameter Tuning Strategy

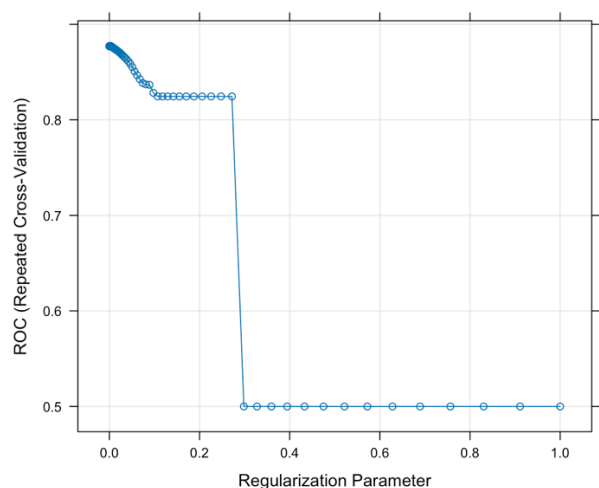
The Logistic LASSO model was tuned with 10-fold cross-validation repeated 3 times to ensure stability. The range of 100 lambda values (ranging from 0.0001 to 1.0) were tested, and alpha was set to 1 for LASSO regularization. The model was optimized for ROC AUC, which is a standard metric for evaluating the performance of binary classifiers.

Cross-Validation Results

Performance was optimal for smaller lambda values. The ROC deteriorated as the penalty increased, demonstrating that excessive shrinkage decreases model accuracy. The optimal lambda value was 0.00163, providing the optimal cross-validated results:

- ROC AUC: 0.877
- Sensitivity: 0.802
- Specificity: 0.799

This blend of performing well and being simple was employed in the final testing.



Interpretation - The model's performance is stable for small `lambda` values but drops sharply as `lambda` increases beyond approximately 0.25. This indicates that while some regularization is beneficial, aggressive penalization (high `lambda`) removes too much information, harming the model's predictive power. The optimal model retains most predictors, applying only a slight penalty to reduce complexity.

Figure 1– ROC against Regularization Parameter Graph

Lambda	Mean ROC (CV Score)	Mean Sensitivity	Mean Specificity
0.0001000000	0.8771	0.8018	0.7975
0.0001097499	0.8771	0.8018	0.7975
0.0001204504	0.8771	0.8018	0.7975
0.0001321941	0.8771	0.8018	0.7975
0.0001450829	0.8771	0.8018	0.7975

Table 1 – Top 5 CV Results – Logistic LASSO Regression

Interpretation - The CV results for the LASSO model show that the highest ROC AUC score (0.877) is achieved with a very small regularization parameter (`lambda` = 0.00163).

Performance Evaluation on Test Set

Test Set Confusion Matrix

Predicted/Actual	Yes	No
Yes	10,598 (Sensitivity: 81.6%)	2,572 (False Positives: 20.0%)
No	2,384 (False Negatives: 18.4%)	10,307 (Specificity: 80.0%)

Table 2 – Confusion matrix – Logistic LASSO Regression

Interpretation - This matrix shows the LASSO model correctly identified 10,598 APT attacks but misclassified 2,384 actual attacks as benign.

Performance Metrics Evaluation

Overall Accuracy (80.8%): The LASSO model was able to accurately predict approximately 81 out of 100 cases in the test set with good and well-balanced results.

Sensitivity (81.6%): This model discovered 81.6% of all actual APT attacks. That is acceptable, yet it is missing nearly one out of every five important attacks, which is a major risk in cybersecurity.

Specificity (80.0%): The model correctly identified 80.0% of all non-APT (benign) traffic. The corresponding 20.0% false positive rate means that a considerable amount of legitimate traffic would be incorrectly flagged for investigation.

Modelling Approach 2: Classification Tree

Hyperparameter Tuning Strategy

With the caret package and fixed seed (10687611) to obtain the same results every time, the tree model was tuned by grid search over the Complexity Parameter (cp), which determines trimming. Repeated 3 times, a 10-fold cross-validation ensured the performance estimate was solid.

Range Tested: 20 cp values ranging from 0 to 0.1 with highly complex trees and highly trimmed trees.

Optimization Metric: ROC AUC, a standard metric for binary classification tasks.

Justification for cp range: The parameter 'cp' is used to balance how complex the tree is and how well the tree fits the data. If we use 0, we obtain a complete tree that fits the data too well. If the parameter is too large, the tree can be too simple. We can experiment with various tree depths by trying out values from 0 to 0.1 to obtain the best balance that performs well with new data, as verified by cross-validation.

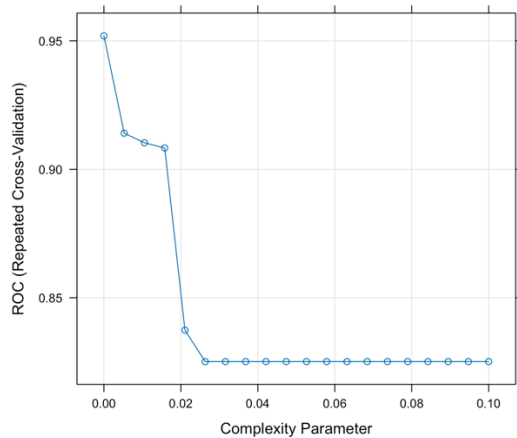
Cross-Validation Results

The best performance was achieved at **cp = 0**, indicating that retaining complexity improved detection. The table below shows the **top 5 CV results**:

cp	Mean ROC	Sensitivity	Specificity
0.0000	0.9519	0.9057	0.9054
0.0053	0.9141	0.8904	0.8765
0.0105	0.9103	0.8919	0.8615
0.0158	0.9083	0.8919	0.8495
0.0211	0.8374	0.9198	0.7504

Table 3 – Top 5 CV Results – Classification Tree

Interpretation - This trend shows that performance declined as cp increased, confirming that complex models were more effective at capturing the patterns in the APT dataset.



Interpretation - Figure 2 clearly illustrates a downward trend: as cp increases, the tree becomes more heavily pruned, and its ability to distinguish between APT and non-APT cases diminishes rapidly. This suggests that a complex, fully-grown tree is necessary to capture the intricate patterns in the data, and any pruning sacrifices significant predictive accuracy.

Figure 2 – ROC against CP Graph

Performance Evaluation on Test Set

Test Set Confusion Matrix

Predicted/Actual	Yes	No
Yes	11,838 (Sensitivity: 91.2%)	1,243 (False Positives: 9.7%)
No	1,144 (False Negatives: 8.8%)	11,636 (Specificity: 90.3%)

Table 4 – Confusion matrix – Classification Tree

Interpretation - The single classification tree correctly identified 11,838 APT attacks and demonstrates a strong ability to distinguish between classes.

Performance Metrics Evaluation

Overall Accuracy (90.8%): The classification tree performs exceptionally well in making predictions with nearly 91% accuracy. That is highly impressive in comparison to the LASSO model.

Sensitivity (91.2%): The model identifies 91.2% of actual APT attacks, and that is excellent. That is to say, it will catch nearly all malicious actions but will leave out approximately 9 out of 100 actual attacks.

Specificity (90.3%): The model is equally strong at identifying benign traffic, with a low false positive rate of 9.7%. This ensures that security alerts are more likely to be meaningful, reducing analyst fatigue from investigating false alarms.

Modelling Approach 3: Bagging Tree

Hyperparameter Tuning Strategy

A grid search was performed to discover the optimal set of hyperparameters. A new bagged model was trained on each set of hyperparameters and measured how well it performs by estimating the Out-of-Bag (OOB) error. The OOB error is a reliable estimate of prediction error under bagging. In each tree, data not in the bootstrap sample is used to make predictions, and the mean error of all the trees is the OOB error. We will select the set of hyperparameters that reduces the OOB misclassification error.

When defining the hyperparameter grid for the grid search, nbagg, cp, and minsplit will be tuned simultaneously with 3 values each.

- nbagg: Number of bootstrap samples (trees).
- cp: Complexity Parameter. Justification: A value of 0 allows trees to grow to their maximum depth. Small positive values (0.01, 0.001) are tested to see if a minor amount of pruning can improve the model by reducing variance.
- minsplit: The minimum number of observations required in a node to attempt a split.

Cross-Validation Results

Optimal OOB Misclassification Error

The best-performing model's Out-of-Bag (OOB) misclassification rate was 0.0745, or 7.45%. This is how often it will be wrong on new data and is computed in training. This is evidence that prior to testing, more than 92.5% of cases were expected to be correctly classified by the bagged model.

Model #	nbagg (Number of Trees)	cp (Complexity Parameter)	minsplit	OOB Error
12	150	0	10	0.0745
3	150	0	5	0.0750
2	100	0	5	0.0757
11	100	0	10	0.0761
1	50	0	5	0.0762

Table 5 - Top 5 CV Results – Bagging Tree

Interpretation - All of the top 5 CV results of the bagged model have a cp of 0, consistent with the conclusion of the single tree that optimal complexity is best. The results are as we would expect: from 50 to 150 trees (nbagg) decreases the OOB error rate (although not in all cases). A minsplit of 10 performed slightly better than with a minsplit of 5, suggesting avoiding splitting on very small sets of observations will assist in generalisation.

Performance Evaluation on Test Set

Test Set Confusion Matrix

Predicted/Actual	Yes	No
Yes	12,028 (Sensitivity: 92.7%)	890 (False Positives: 6.9%)
No	954 (False Negatives: 7.3%)	11,989 (Specificity: 93.1%)

Table 6 – Confusion matrix – Bagging Tree

Interpretation - The bagging model shows the highest number of correctly identified APT attacks (12,028) and the lowest number of false classifications.

Performance Metrics Evaluation

Overall Accuracy (92.9%): The bagging model achieves the highest accuracy of all three models at 92.9%. This ensemble method demonstrates superior predictive capability by effectively reducing the variance of a single decision tree.

Sensitivity (92.7%): It identifies more APTs than any other technique. It identifies 92.7% of actual attacks correctly and is best at detecting the attackers, missing approximately 7 out of 100 attacks.

Specificity (93.1%): It is extremely good at disregarding benign traffic with a high specificity of 93.1%. Due to this, it has the lowest false positive rate (6.9%), which makes it extremely suitable for a security operations center in real life.

Recommended Model and Conclusion

Comparative Performance Analysis

Performance Metric	LASSO Regression	Classification Tree	Bagging Tree
Overall Accuracy	80.8%	90.8%	92.9%
Sensitivity (APT Detection)	81.6%	91.2%	92.7%
Specificity (Non-APT Detection)	80.0%	90.3%	93.1%
False Positive Rate	20.0%	9.7%	6.9%
False Negative Rate	18.4%	8.8%	7.3%

Table 7 – Comparing 3 ML Models.

Justification for the Recommended Model

While the Classification Tree was better than LASSO Regression, the **Bagging Tree** is by far better on all the key performance measures. It is the best in Overall Accuracy (92.9%), meaning that it makes the best predictions.

Most significantly in the security context, it has the best Sensitivity (92.7%), that is, it maximizes the detection of true APT attacks and minimizes the possibility of missing an attack (lowest False Negative Rate of 7.3%). Additionally, with the best Specificity (93.1%), security resources are not wasted in pursuit of innumerable false alarms (lowest False Positive Rate of 6.9%). Through averaging predictions from many trees, bagging effectively reduces the model's variance and provides a more stable and reliable solution to catch sophisticated threats.

Conclusion

This analysis tested three machine learning models to detect APT attacks. Bagging Tree excelled, beating LASSO Regression and the individual Classification Tree with a detection rate of 92.9% on the test dataset and increased sensitivity and specificity. Its excellent detection with few false alarms make it the best-fitting model to this cybersecurity application.

References

Dremio. (2024, July 25). *What is Out-of-Bag Error?* | Dremio.

<https://www.dremio.com/wiki/out-of-bag-error/>

Great Learning. (2024, October 14). *A Complete understanding of LASSO Regression*. Great Learning Blog: Free Resources What Matters to Shape Your Career!

<https://www.mygreatlearning.com/blog/understanding-of-lasso-regression/>

Shishodiya, I. (2022b, March 26). Decision trees, bootstrap aggregating and bagging. - Ishan

Shishodiya - medium. *Medium*. <https://medium.com/@sly.of.zero/decision-trees-bootstrap-aggregating-and-bagging-8c6cf764e689>

Appendices

Appendix A: Model Selection Console Output

```
Console Terminal Background Jobs
R 4.4.2 ~/Documents/Assignment_3_ML/

> library(tidyverse)
>
> # Setting seed using my student ID
> set.seed(10687611)
>
> # Define the available model options
> models.list1 <- c("Logistic Ridge Regression",
+                  "Logistic LASSO Regression",
+                  "Logistic Elastic-Net Regression")
>
> models.list2 <- c("Classification Tree",
+                  "Bagging Tree",
+                  "Random Forest")
>
> # Randomly sample 1 from list1 and 2 from list2
> myModels <- c(sample(models.list1, size = 1),
+               sample(models.list2, size = 2))
>
> # Display the selected models
> myModels %>% data.frame()

1 Logistic LASSO Regression
2 Classification Tree
3 Bagging Tree
>
```

Appendix B: Confusion Matrix and Statistics Console Output – LASSO Regression

```
1 # Model 2- Classification Tree
2
3 library(tidyverse)
4
18:51 (Top Level)
Console Terminal Background Jobs
R 4.4.2 ~/Documents/Assignment_3_ML/

--- Confusion Matrix and Performance Metrics (Test Set) ---
>
> predictions <- factor(predictions, levels = levels(test_set[[outcome_col_name]]))
>
> confusion_matrix_results <- confusionMatrix(
+   data = predictions,
+   reference = test_set[[outcome_col_name]],
+   positive = "Yes" # Define the "positive" class for correct metric calculation
+ )
> print(confusion_matrix_results)
Confusion Matrix and Statistics

              Reference
Prediction Yes No
Yes 10598 2572
No 2384 10307

              Accuracy : 0.8084
              95% CI : (0.8035, 0.8131)
              No Information Rate : 0.502
              P-Value [Acc > NIR] : <2e-16

              Kappa : 0.6167

McNemar's Test P-Value : 0.0079

              Sensitivity : 0.8164
              Specificity : 0.8003
              Pos Pred Value : 0.8047
              Neg Pred Value : 0.8122
              Prevalence : 0.5020
              Detection Rate : 0.4098
              Detection Prevalence : 0.5093
              Balanced Accuracy : 0.8083

              'Positive' Class : Yes
>
```

Appendix C: Confusion Matrix and Statistics Console Output – Classification Tree

```
3 library(tidyverse)
4
86:24 (Top Level)
Console Terminal Background Jobs
R 4.4.2 ~/Documents/Assignment_3_ML/

--- Confusion Matrix and Performance Metrics (Test Set) ---
> tree_confusion_matrix <- confusionMatrix(
+   data = tree_predictions,
+   reference = test_set[[outcome_col_name]],
+   positive = "Yes"
+ )
> print(tree_confusion_matrix)
Confusion Matrix and Statistics

              Reference
Prediction Yes No
Yes 11838 1243
No 1144 11636

              Accuracy : 0.9077
              95% CI : (0.9041, 0.9112)
              No Information Rate : 0.502
              P-Value [Acc > NIR] : < 2e-16

              Kappa : 0.8154

McNemar's Test P-Value : 0.04487

              Sensitivity : 0.9119
              Specificity : 0.9035
              Pos Pred Value : 0.9050
              Neg Pred Value : 0.9105
              Prevalence : 0.5020
              Detection Rate : 0.4578
              Detection Prevalence : 0.5058
              Balanced Accuracy : 0.9077

              'Positive' Class : Yes
>
```

Appendix D: Confusion Matrix and Statistics Console Output – Bagging Tree

```
1 #-----
2 # MAT3120.3 - Assignment 3: Bagging Model
3 # Student ID: 10687611
4
14:14 (Untitled)
Console Terminal Background Jobs
R 4.4.2 ~/Documents/Assignment_3_ML/

--- Confusion Matrix and Performance Metrics for Bagging (Test Set) ---
> # Generate and display the confusion matrix and associated statistics.
> confusion_matrix_bagging <- confusionMatrix(
+   data = predictions_bagging,
+   reference = test_set[[outcome_col_name]],
+   positive = "Yes" # Define the "positive" class for correct metric calculation
+ )
> print(confusion_matrix_bagging)
Confusion Matrix and Statistics

              Reference
Prediction Yes No
Yes 12028 890
No 954 11989

              Accuracy : 0.9287
              95% CI : (0.9255, 0.9318)
              No Information Rate : 0.502
              P-Value [Acc > NIR] : <2e-16

              Kappa : 0.8574

McNemar's Test P-Value : 0.1423

              Sensitivity : 0.9265
              Specificity : 0.9309
              Pos Pred Value : 0.9311
              Neg Pred Value : 0.9263
              Prevalence : 0.5020
              Detection Rate : 0.4651
              Detection Prevalence : 0.4995
              Balanced Accuracy : 0.9287

              'Positive' Class : Yes
>
```